DOUBLY LINKED LIST

1. Introduction

A doubly linked list (DLL) is a linear data structure made of nodes. Each node stores data, a pointer to the previous node (prev), and a pointer to the next node (next). It supports movement in both directions.

Example: NULL <- 10 <-> 20 <-> 30 -> NULL. The first node has prev = NULL; the last node has next = NULL. Head points to the first node.

2. Node structure in C++

```
struct Node {
    int data;
    Node* prev;
    Node* next;

    Node(int val) {
        data = val;
        prev = NULL;
        next = NULL;
    }
};
```

The next pointer moves forward; prev moves backward. The extra prev pointer uses more memory than a singly linked list, but makes backward traversal possible and simplifies deletion when a node pointer is already known.

3. Traversal

Forward traversal starts at head and follows next until NULL. Backward traversal starts at the tail and follows prev until NULL.

```
void printForward(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
}

void printBackward(Node* tail) {
    Node* temp = tail;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->prev;
    }
}
```

For 10 <-> 20 <-> 30, forward output is 10 20 30 and backward output is 30 20 10.

4. Insertion

4.1 Insert at head

Create the new node.

Point newNode->next to the current head.

If the list is not empty, set the old head's prev to the new node.

Update head to newNode. Its prev is already NULL.

```
void insertAtHead(Node*& head, int val) {
    Node* newNode = new Node(val);
    newNode->next = head;

    if (head != NULL)
        head->prev = newNode;

    head = newNode;
}
```

Time: O(1). Example: 10 <-> 20 becomes 5 <-> 10 <-> 20.

4.2 Insert at tail

Without a tail pointer, traverse to the last node. Link the old last node to the new node and set the new node's prev to the old last node.

```
void insertAtTail(Node*& head, int val) {
    Node* newNode = new Node(val);

    if (head == NULL) {
        head = newNode;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}
```

Time: O(n) without a tail pointer. If a tail pointer is maintained, insertion at tail is O(1).

4.3 Insert after a given node

If ptr points to the node after which we insert, save its successor first, then connect all four relevant links.

```
void insertAfter(Node* ptr, int val) {
    if (ptr == NULL) return;

    Node* newNode = new Node(val);
    Node* successor = ptr->next;

    newNode->prev = ptr;
    newNode->next = successor;
    ptr->next = newNode;
```

```
    if (successor != NULL)
        successor->prev = newNode;
}
```

Time: O(1) if ptr is already known. Searching for the node first may take O(n).

5. Deletion

5.1 Delete head

```
void deleteHead(Node*& head) {
    if (head == NULL) return;

    Node* oldHead = head;
    head = head->next;

    if (head != NULL)
        head->prev = NULL;

    delete oldHead;
}
```

Handles an empty list, a one-node list (head becomes NULL), and a multi-node list. Time: O(1).

5.2 Delete tail

```
void deleteTail(Node*& head) {
    if (head == NULL) return;

    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->prev->next = NULL;
    delete temp;
}
```

Stop at the last node. Its previous node becomes the new tail, so set that node's next to NULL. Time: O(n) without a tail pointer; O(1) with a maintained tail pointer.

5.3 Delete a known node

```
void deleteNode(Node*& head, Node* ptr) {
    if (head == NULL || ptr == NULL) return;

    if (ptr == head)
        head = ptr->next;

    if (ptr->next != NULL)
        ptr->next->prev = ptr->prev;

    if (ptr->prev != NULL)
        ptr->prev->next = ptr->next;
```

```
    delete ptr;
}
```

Time: O(1) when ptr is already known; O(n) overall if we first search for a value.

Operation	Complexity	Description
Insertion at Head	`O(1)`	Add new node at beginning, update head and adjacent node's pointers
Insertion at Tail	`O(1)`	Add new node at end using tail pointer
Insertion at Position	`O(n)`	Traverse to position and insert with pointer updates
Deletion at Head	`O(1)`	Remove first node and update head pointer
Deletion at Tail	`O(1)`	Remove last node using tail pointer
Deletion by Value	`O(n)`	Traverse to find node and update adjacent pointers
Forward Traversal	`O(n)`	Traverse from head to tail using next pointers
Backward Traversal	`O(n)`	Traverse from tail to head using prev pointers

TREES & BINARY TREES

What is a tree?

A tree is a non-linear hierarchical data structure made of nodes connected by edges. It branches rather than storing elements in one straight sequence. Common uses include folder structures, organization charts and expression trees.

A non-empty tree with N nodes has N - 1 edges.

Tree terminology

Term	Meaning
Root	Topmost node; it has no parent.
Parent	A node that has one or more children.
Child	A node directly below another node.
Siblings	Nodes with the same parent.
Leaf	A node with no children.
Internal node	A node with at least one child.
Edge	A connection between two nodes.
Subtree	A node together with its descendants.
Depth	Number of edges from root to that node.
Height of node	Number of edges in its longest downward path to a leaf.
Height of tree	Height of the root.

With height measured in edges, a single-node tree has height 0. Some materials measure height in nodes/levels; check the convention in each question.

Binary tree introduction

A binary tree is a tree in which each node has at most two children: a left child and a right child. A node may have zero, one or two children. Left and right positions are distinct.



Here, 1 is the root; 4, 5 and 6 are leaves; 2 is the parent of 4 and 5. The subtree rooted at 2 contains 2, 4 and 5.

Basic types of binary trees

Type	Meaning
Full / strict	Every node has either 0 or 2 children.
Complete	All levels are full except possibly the last; the last level fills from left to right.
Perfect	Every internal node has exactly 2 children and all leaves are at the same level.
Skewed	Each node has only one child, making a chain-like tree.

Balanced	Left and right subtree heights remain reasonably close; precise definitions depend on the tree type.

Binary tree node in C++

```
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = NULL;
        right = NULL;
    }
};
```

Each node stores a value and pointers to its left and right children. A missing child is represented by NULL (or nullptr).

Tree traversal

Traversal means visiting every node in a defined order. Preorder, inorder and postorder are depth-first traversals (DFS). Their recursive implementations differ only in when the current/root node is processed.

Preorder — Root, Left, Right

Visit the root first, then traverse its left subtree, then its right subtree.

```
void preorder(Node* root) {
    if (root == NULL) return;

    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}
```

For the sample tree: 1 2 4 5 3 6. Memory cue: PREorder processes the root before its subtrees.

Inorder — Left, Root, Right

Traverse the left subtree, visit the root, then traverse the right subtree.

```
void inorder(Node* root) {
    if (root == NULL) return;

    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

For the sample tree: 4 2 5 1 3 6. Important: inorder traversal of a Binary Search Tree (BST) gives sorted values. Inorder does not necessarily sort an arbitrary binary tree.

Postorder — Left, Right, Root

Traverse the left subtree, then the right subtree, and visit the root at the end.

```
void postorder(Node* root) {
    if (root == NULL) return;

    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

For the sample tree: 4 5 2 6 3 1. Memory cue: POSTorder processes the root after its subtrees.

Compare the three DFS traversals

Traversal	Visit order	Sample output
Preorder	Root → Left → Right	1 2 4 5 3 6
Inorder	Left → Root → Right	4 2 5 1 3 6
Postorder	Left → Right → Root	4 5 2 6 3 1

The tree does not change. Only the order in which nodes are visited changes.

Level Order Traversal (BFS)

Level order visits nodes level by level, from left to right within each level. It is also called Breadth-First Search (BFS). The standard implementation uses a queue.

For the sample tree: level 0 = 1; level 1 = 2, 3; level 2 = 4, 5, 6. Output: 1 2 3 4 5 6.

1. Why a queue?

A queue follows FIFO (First In, First Out). We remove the earliest-added node, print it, and add its children. This processes nodes in breadth-first, level-by-level order.

2. C++ implementation

```
#include <queue>

void levelOrder(Node* root) {
    if (root == NULL) return;

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* curr = q.front();
        q.pop();

        cout << curr->data << " ";

        if (curr->left != NULL)
            q.push(curr->left);

        if (curr->right != NULL)
            q.push(curr->right);
    }
}
```

3. Queue dry run

Step	Print/remove	Children added	Queue after step
Start	—	1	[1]
1	1	2, 3	[2, 3]

2	2	4, 5	[3, 4, 5]
3	3	6	[4, 5, 6]
4	4	None	[5, 6]
5	5	None	[6]
6	6	None	[]

When the queue becomes empty, traversal is complete. Printed order: 1 2 3 4 5 6.

Complexity of traversals

Traversal	Time	Auxiliary space
Preorder	O(n)	O(h) recursion stack
Inorder	O(n)	O(h) recursion stack
Postorder	O(n)	O(h) recursion stack
Level order	O(n)	O(w) queue, w = maximum width

n = number of nodes; h = tree height; w = maximum number of nodes at any one level. A skewed tree can make recursive stack space O(n); a balanced tree generally has height O(log n). The level-order queue can hold O(n) nodes in the widest level.

Complete traversal program

```
#include <iostream>
#include <queue>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) {
        data = val;
        left = NULL;
        right = NULL;
    }
};

void preorder(Node* root) {
    if (root == NULL) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

void postorder(Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

```
void levelOrder(Node* root) {
    if (root == NULL) return;
    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* curr = q.front();
        q.pop();
        cout << curr->data << " ";

        if (curr->left != NULL) q.push(curr->left);
        if (curr->right != NULL) q.push(curr->right);
    }
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->right->right = new Node(6);

    cout << "Preorder: "; preorder(root);
    cout << "\nInorder: "; inorder(root);
    cout << "\nPostorder: "; postorder(root);
    cout << "\nLevel order: "; levelOrder(root);
    return 0;
}
```

Expected output:

```
Preorder: 1 2 4 5 3 6
Inorder: 4 2 5 1 3 6
Postorder: 4 5 2 6 3 1
Level order: 1 2 3 4 5 6
```